Daniil Shykhov
ainative.to

Beginner's Guide to RAG

What I wish I knew before building my first one.

THE CONCEPT

RAG is an open-book exam for your AI.

Without RAG, your AI answers from memory — whatever it was trained on. It doesn't know your docs, your codebase, or your company wiki.

With RAG, you hand it the relevant pages before it answers. It reads them, then responds using **your data**.

The acronym tells you exactly what happens:

- **Retrieval** — find the right documents
- **Augmented** — add them to the prompt
- **Generation** — the AI answers using those documents

THE 5-MINUTE VERSION

RAG = "search your docs, paste the results into the prompt, let the AI answer." Everything else is implementation details.

BEFORE YOU START

Do you even need RAG?

Context windows are huge now. Claude handles 200K tokens. GPT-4o does 128K. That's hundreds of pages.

Ask yourself these questions first:

JUST PASTE IT IN THE PROMPT

Your data fits in the context window (a few docs, one codebase, a wiki section). Start here. If it works, you're done.

YOU NEED RAG WHEN

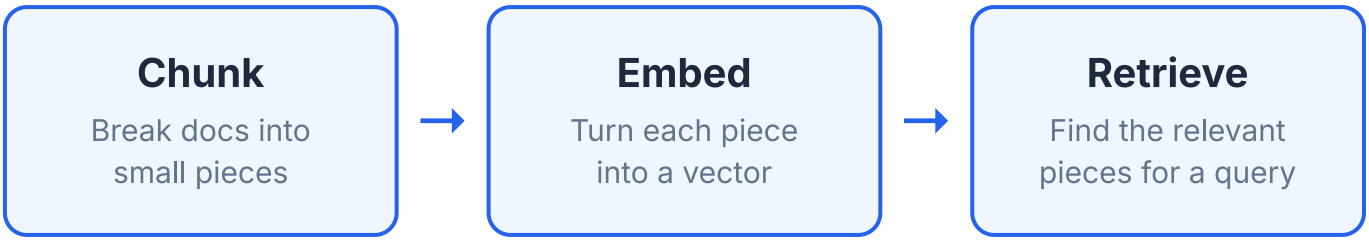
Your data is too big for the context window. Or it changes frequently. Or you need to cite which specific document the answer came from.

REAL TALK

I've seen teams build full RAG pipelines for 20 pages of docs that fit in a single prompt. Try the simple approach first. Build RAG when you hit the wall.

HOW IT WORKS

Three steps.
That's the whole pipeline.



When someone asks a question, you search your stored vectors for the most relevant chunks. Then you paste those chunks into the AI's prompt alongside the question.

The AI reads your chunks and generates an answer. That's it.

You do the chunking and embedding once (when you load your documents). Retrieval and generation happen every time someone asks a question.

STEP 1: CHUNKING

Chunk size is the lever most people ignore.

Chunking is splitting your documents into smaller pieces. The size of those pieces changes everything about your results.

CHUNK SIZE	GOOD FOR	TRADE-OFF
`~200 tokens`	Precise Q&A, specific facts	More chunks to search, less context per chunk
`~500 tokens`	General knowledge bases	Balance of precision and context
`~1000 tokens`	Summaries, long-form docs	Less precise retrieval, but more context per result

Start with 200. You can always go bigger. Going smaller later means re-processing all your documents from scratch.

TIP

Overlap your chunks by 10-20%. If chunk 1 ends mid-sentence, chunk 2 should start a few sentences earlier. This prevents splitting an idea across two chunks.

STEP 2: EMBEDDINGS

Pick a model. Don't overthink it.

Embeddings turn text into numbers (vectors) so you can search by meaning instead of keywords. "How do I get a refund?" matches "return policy" even though they share no words.

MODEL	PROVIDER	NOTES
`text-embedding-3-small`	OpenAI	Cheap, good enough for most use cases
`text-embedding-3-large`	OpenAI	Better accuracy, 6x the cost
`embed-v4`	Cohere	Strong multilingual support
`all-MiniLM-L6-v2`	Open source	Free, runs locally, decent quality

HONEST TAKE

Your embedding model matters less than your chunk size and retrieval strategy. Pick `text-embedding-3-small` and move on. You can swap it later.

STEP 3: STORAGE

Where to store your vectors.

A vector database stores your embeddings and lets you search them by similarity. Pick one based on where you are:

DATABASE	BEST FOR	SETUP
`Chroma`	Getting started, prototypes	`pip install`, runs locally
`Pinecone`	Production, managed	API key, zero ops
`pgvector`	Already using Postgres	Extension, no new infra
`Qdrant`	Self-hosted, high perf	Docker, open source

WHERE TO START

Use Chroma. It runs locally, needs zero config, and you can switch to Pinecone or pgvector later when you need to.

THE HARD PART

Bad retrieval = bad answers. Every time.

Your AI can only answer well if it receives the right chunks. This is where most RAG systems fail — not the model, not the prompt.

BAD RETRIEVAL

Question: "What's our refund policy?"
Retrieved chunk: "Our company was founded in 2015 with a mission to..."
AI answer: Makes something up. Hallucination.

GOOD RETRIEVAL

Question: "What's our refund policy?"
Retrieved chunk: "Refunds are available within 30 days of purchase..."
AI answer: Uses the actual policy. Accurate.

If your AI is hallucinating, don't blame the model. **Check what chunks it actually received.** Nine times out of ten, it's a retrieval problem.

TESTING

Test with questions you already know the answer to.

This is the simplest debugging method and most people skip it.

1. Pick **10 questions** your documents can answer. You need to know the correct answers yourself.
2. Run them through your RAG pipeline. Save the answers **and** the retrieved chunks.
3. **Wrong answer + right chunks?** That's a prompt problem. Rewrite how you ask the AI to use the context.
4. **Wrong answer + wrong chunks?** That's a retrieval problem. Fix your chunk size, overlap, or embedding model.

KEY INSIGHT

Most RAG failures are retrieval failures. The AI is fine — it's just reading the wrong pages.

WHERE TO START

The simplest RAG stack that actually works.

Option A: Managed (fastest start)

OpenAI Assistants API with file search. Upload your files, ask questions. It handles chunking, embeddings, and retrieval for you. Zero pipeline code.

Option B: Build it yourself (more control)

```
LlamaIndex + Chroma + OpenAI embeddings
~50 lines of Python
You control chunk size, retrieval, and prompts
```

Start with Option A. It gets you a working RAG system in minutes. Move to Option B when you need to control chunk sizes, swap models, or debug retrieval.

DON'T DO THIS

Don't hand-roll embeddings, build your own vector search, or write a chunking algorithm on day one. Use existing tools. Build from scratch when you know what's breaking.

TRY THIS NOW

Build your first RAG in 15 minutes.

Stop reading. Build something.

1. **Pick one document** — internal docs, a long PDF, your company wiki. Something you know well enough to test.
2. Go to **OpenAI Playground** → Assistants → Create → enable **File Search**.
3. **Upload your document.** OpenAI chunks and embeds it automatically.
4. **Ask 5 questions you know the answer to.** Does it get them right? Does it cite the right sections?
5. **Find a wrong answer.** That's your first retrieval problem. Now you know what to fix.

That's RAG. You just built one. Now you know what to improve.

Want the next guide?

Practical AI guides for engineers. No hype, no fluff. Just what actually works.

Every week. Free.

ainative.to

Daniil Shykhov